

FPGA Board Bring-Up and Verification

RISC-V Automotive ECU SoC | Nexys 4 DDR (XC7A100T-1CSG324C) | 80 MHz

What this document is for

The bitstream is built and verified in simulation and post-route timing. This is the procedure for programming the board and confirming the SoC works on real silicon. It covers what to press, what to look at, and what each outcome means.

The whole test reduces to one thing: **the number 0x245 appearing on LED[15:0]**. Everything below explains how to get there and what to do if you don't.

What the test actually proves

The program on the board makes the CPU configure the CAN controller over the peripheral bus, transmit one CAN frame (ID 0x245, data DEADBEEF 12345678, DLC 8), and because the controller is in loopback the frame is serialised onto the wire and decoded straight back. That raises CAN RX, which is IRQ 0 and the highest priority source in the system. The interrupt reaches the CPU through the interrupt controller, the trap handler reads the received registers, and writes the ID to the GPIO output register.

So if 0x245 lights up, all of the following worked at once:

Subsystem	What 0x245 proves about it
Clock generator	MMCM locked and the SoC is running at 80 MHz
Instruction memory	program.mem was loaded into the bitstream and is executing
Bus interconnect	Address decoding routes to GPIO, CAN and INTC correctly
CAN controller	Bit timing, framing, stuffing, CRC and FIFOs all function
Interrupt controller	Latched the source, resolved priority, drove the CPU line
CPU trap logic	mtvec, mcause, mepc and mret all behave
GPIO	Register writes reach the pins

No CAN transceiver needed for this test

The controller's loopback bit routes the transmit wire straight back to the receiver inside the FPGA. Pmod JD can be left empty. A transceiver is only needed later, when loopback is cleared and you want to talk to a real CAN bus.

Part 1 / Programming the board

Before you plug anything in

Set all 16 slide switches to the **down (off)** position. The GPIO inputs read the switches. The test program doesn't use them, but starting from a known state removes one variable if something looks odd.

1 Connect the board

Micro-USB cable from your PC to the **PROG** port (the one nearest the power switch, labelled J6). This carries both power and JTAG.

Slide the power switch to **ON**. The red power LED lights and you may see whatever demo bitstream shipped on the board start running. That's expected.

2 Open the Hardware Manager

In Vivado, Flow Navigator, bottom of the list: **PROGRAM AND DEBUG** → **Open Hardware Manager**.

A green banner appears across the top of the window: *No hardware target is open. Open target.* Click **Open target** → **Auto Connect**.

After a few seconds the Hardware panel should show:

localhost (1) → xilinx_tcf/Digilent/2103xxxxxx → **xc7a100t_0**

If Auto Connect finds nothing

Check the power switch is on and the cable is in the PROG port, not the OTG port.

On Windows, Digilent drivers install with Vivado but occasionally need the board unplugged and replugged once.

Try a different USB port. Front-panel hubs are a common culprit.

3 Program the device

Right-click **xc7a100t_0** → **Program Device...**

The Bitstream File field should already point at:

<project>.runs/impl_1/nexys4_top.bit

If it's empty or points somewhere else, browse to that path. Leave the Debug Probes field blank — you have no ILA in this build.

Click **Program**. It takes a few seconds. The **DONE** LED on the board (near the FPGA, labelled LD20 or DONE) turns on.

Programming is volatile

This writes directly to the FPGA and is lost on power-off. That is fine for testing. To make it permanent you would generate a memory configuration file and write it to the QSPI flash, which is not needed here.

Part 2 / Reading the result

Look at the board immediately after programming. The interrupt LED flashes once, about a fifth of a second, and then the board sits idle. If you miss it, press **BTNC** (the centre button of the five-button cluster) to reset and watch again.

Status LEDs

Four coloured LEDs sit above the main row, to the right. They report the health of the SoC itself.

LED	Colour / pin	Expected	Meaning
LED16	Blue, R12	ON steady	MMCM locked, 80 MHz clock is stable
LED17	Green, M16	ON steady	SoC released from reset and running
LED18	Red, N15	One flash	A CAN RX interrupt was taken
LED19	Blue, G14	OFF	No unmapped bus access. This one must stay dark.

The main LED row — this is the actual result

LED[15:0] holds the CAN ID received back through loopback. It should read **0x245**, which in binary across the row is:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	1	0	0	1	0	0	0	1	0	1

Reading right to left from LED0: three lit LEDs at positions **0, 2, 6** and one at **9**. Nothing else. That is 0x245.

If you see that pattern, the project works.

The CPU, bus, interrupt controller, GPIO and Person 1's CAN controller are all functioning together on hardware. Take a photograph of the board for your report — it is the single clearest piece of evidence you have.

Press BTNC and it repeats

BTNC is the reset. Every press restarts the program: the LEDs clear, the controller is reconfigured, a new frame goes out, LED18 flashes, and 0x245 reappears. Repeat it a few times — consistent behaviour across resets is worth confirming.

Part 3 / If it doesn't work

Each failure pattern points at a different subsystem. Find the row that matches what you see.

What you see	What it means	What to do
Everything dark. Even DONE is off.	The bitstream never loaded.	Re-run Program Device. Check the cable is in the PROG port.
DONE on, LED16 off.	The clock generator is not locking. Nothing downstream can run.	Confirm clk_wiz_0 was regenerated for 80.000 MHz with the <i>locked</i> output enabled. Check CLK100MHZ is constrained to pin E3.
LED16 on, LED17 off.	The SoC is stuck in reset.	BTNC may be held or stuck. Press and release it firmly. If it stays off, check BTNC is on pin N17 and not inverted.
LED16 and LED17 on, everything else dark.	The CPU is clocked and running but nothing happened. Most likely the program never made it into the bitstream.	Search the <i>synthesis</i> log for readmemh. If the file was not found, the instruction memory is full of NOPs. Add program.mem to Design Sources , not just Simulation Sources, and rebuild.
LED19 lit.	Software touched an unmapped address. With this program that should be impossible.	Points to a corrupt or wrong program.mem. Confirm the first line of the file is 400000B7.
LED18 flashes but the LEDs are wrong.	The interrupt path works. The received CAN data does not.	The most interesting failure. See the ILA section below.
LED18 never flashes, LEDs stay at 0.	No interrupt. The CAN frame is not completing.	Almost always the bit timing not matching the clock. Re-check that clk_wiz_0 really outputs 80 MHz — run <code>report_clocks</code> on the implemented design and look for 12.500 ns.

Adding an ILA, only if you need one

The debug nets are already tagged with `mark_debug` in `nexys4_top.v`, so they survive synthesis and can be selected directly. Adding a debug core costs a full rebuild of roughly twenty minutes and consumes BRAM, so do it only when the LEDs cannot distinguish between two possible causes.

Flow Navigator → **SYNTHESIS** → **Set Up Debug**. Add these nets:

Net	What it tells you
<code>debug_pc</code>	Where the CPU is executing. Answers almost every question on its own.
<code>debug_instruction</code>	What it is fetching. Confirms the program loaded correctly.
<code>debug_irq_taken</code>	Whether the interrupt fired at all. Use it as the trigger.
<code>bus_error</code>	Whether an unmapped access occurred and at what point.

Rebuild, reprogram, then in the Hardware Manager set the trigger to `debug_irq_taken == 1` and arm it. Reading `debug_pc` alone usually settles it: if it cycles around 0x68 and 0x6C the CPU reached the idle loop and the problem is in the CAN frame; if it sits at 0 or wanders unpredictably, the program did not load.

Part 4 / Going further

Using a real CAN bus

Loopback proves the controller works. To reach a real bus you need a transceiver (MCP2551 or SN65HVD230) on Pmod JD, clear the loopback bit in CAN_CTRL, and set a bit rate that matches the other nodes. The test program uses BRP = 0, which is only valid in loopback. For a real bus at 80 MHz with 7 time quanta per bit:

Bit rate	BRP value in CAN_BTIME[7:0]
125 kbit/s	90
250 kbit/s	44
500 kbit/s	22
1 Mbit/s	10

Pmod JD: JD1 (pin H4) is CAN RX into the FPGA, JD2 (pin H1) is CAN TX out. Wire JD1 to the transceiver's RXD and JD2 to its TXD, plus 3V3 and ground.

Testing the other peripherals

Peripheral	Where it comes out	How to exercise it
GPIO	SW[15:0] in, LED[15:0] out	Already used by the test program. Write a variant that copies switches to LEDs.
UART	USB serial bridge	Open a terminal at 115200 8-N-1 on the board's COM port and have the program send bytes.
PWM	Pmod JB, pins 1-4	Scope it, or wire an LED through a resistor and vary the duty register.
SPI	Pmod JA	Needs a device. Or short MOSI to MISO for a loopback check.
I2C	Pmod JC	Needs a device and external 4.7k pull-ups to 3V3 . The internal pull-ups in the XDC are far too weak for real I2C rise times.
ADC	Not wired	The XADC needs a DRP wrapper that does not exist yet. The driver's busy and done signals never fire, so software must not poll it.

Reference: verified results to date

Check	Result
tb_periph_subsystem (bus, INTC, 7 peripherals, CAN loopback)	ALL CHECKS PASSED
tb_top_level_riscv (CPU, peripherals, timer interrupts)	PASSED, no regression
Post-route timing at 80 MHz	WNS +0.309 ns, 0 failing endpoints of 10920
Utilisation	3896 LUT (6%), 3425 FF, 4 DSP, 1 BRAM, 55 pins
Power estimate	0.149 W

Known deviation, documented: can_regfile.v drives its read data combinationally, one cycle earlier than the SoC bus contract requires. can_adapter.v bridges this at the boundary and the conversion is verified through the real interconnect, including back-to-back reads and the pop-on-read side effect on CAN_RX_CTRL. The proper fix is a clocked read mux in can_regfile.v, at which point REGISTER_RDATA can be set to 0 and the shim becomes a pass-through.